



CS207ES: DATA STRUCTURES LAB MANUAL

Prerequisites: 1. A Course on "Programming for problem solving".

Course Objectives:

1. It covers various concepts of C programming language
2. It introduces searching and sorting algorithms
3. It provides an understanding of data structures such as stacks and queues.

Course Outcomes:

1. Ability to develop C programs for computing and real-life applications using basic elements like control statements, arrays, functions, pointers and strings, and data structures like stacks, queues and linked lists.
2. Ability to Implement searching and sorting algorithms

List of Experiments

1. Operations on Singly Linked List (Creation, Insertion, Deletion, Traversal).
2. Operations on Doubly Linked List (Creation, Insertion, Deletion, Traversal).
3. Operations on Circular Linked List (Creation, Insertion, Deletion, Traversal).
4. Stack implementation using Arrays and ADT.
5. Queue implementation using Arrays and ADT.
6. Sorting Techniques: Radix Sort, Heap Sort, Shell Sort, Tree Sort.
7. Tree Traversal Methods (Recursive and Non-Recursive).
8. Binary Search Tree, B-Trees, B+ Trees, AVL Trees, Red-Black Trees.
9. Graph Traversal Methods.



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)

NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE

Approved by AICTE, New Delhi, Affiliated to JNTUH

10. Hashing Techniques: Division, Multiplication, Mid-Square, Folding.



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

1. C Program to perform Operations on Singly Linked List (Creation, Insertion, Deletion, Traversal).

```
#include<stdio.h>
#include<stdlib.h>
struct node{
    int data;
    struct node *next;
};
struct node *head=NULL;
void create(){
    int n,i;
    printf("Enter number of nodes: ");
    scanf("%d",&n);
    struct node *temp,*newnode;
    for(i=0;i<n;i++){
        newnode=(struct node*)malloc(sizeof(struct node));
        printf("Enter data: ");
        scanf("%d",&newnode->data);
        newnode->next=NULL;
        if(head==NULL)
            head=temp=newnode;
        else{
            temp->next=newnode;
            temp=newnode;    }
    }
}
void insert(){
    struct node *newnode=(struct node*)malloc(sizeof(struct node));
    printf("Enter value: ");
    scanf("%d",&newnode->data);
    newnode->next=head;
    head=newnode;
}
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
void delete_node(){
    int key;
    printf("Enter value to delete: ");
    scanf("%d",&key);
    struct node *temp=head,*prev=NULL;
    while(temp && temp->data!=key){
        prev=temp;
        temp=temp->next;
    }
```

```
    if(!temp) return;
    if(prev==NULL)
        head=temp->next;
    else
        prev->next=temp->next;
    free(temp);
}
```

```
void traverse(){
    struct node *temp=head;
    while(temp){
        printf("%d -> ",temp->data);
        temp=temp->next; }
    printf("NULL\n");
}
```

Output/Results:



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)

NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE

Approved by AICTE, New Delhi, Affiliated to JNTUH

2. C Program to perform Operations on Doubly Linked List (Creation, Insertion, Deletion, Traversal).

```
#include<stdlib.h>
```

```
struct node{  
int data;  
struct node *prev,*next;  
};
```

```
struct node *head=NULL;
```

```
void insert(){  
struct node *newnode=(struct node*)malloc(sizeof(struct node));  
printf("Enter value: ");  
scanf("%d",&newnode->data);
```

```
newnode->prev=NULL;  
newnode->next=head;
```

```
if(head!=NULL)  
head->prev=newnode;
```

```
head=newnode;  
}
```

```
void delete_node(){  
int key;  
printf("Enter value to delete: ");  
scanf("%d",&key);
```

```
struct node *temp=head;
```

```
while(temp && temp->data!=key)  
temp=temp->next;
```

```
if(!temp) return;
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
if(temp->prev)
temp->prev->next=temp->next;
else
head=temp->next;
```

```
if(temp->next)
temp->next->prev=temp->prev;
```

```
free(temp);
}
```

```
void traverse(){
struct node *temp=head;
while(temp){
printf("%d <-> ",temp->data);
temp=temp->next;
}
}
```

Output/Results:



3. C Program to perform Operations on Circular Linked List (Creation, Insertion, Deletion, Traversal).

```
#include<stdlib.h>
```

```
struct node{  
int data;  
struct node *next;  
};
```

```
struct node *last=NULL;
```

```
void insert(){  
struct node *newnode=(struct node*)malloc(sizeof(struct node));  
printf("Enter value: ");  
scanf("%d",&newnode->data);
```

```
if(last==NULL){  
last=newnode;  
last->next=last;  
}else{  
newnode->next=last->next;  
last->next=newnode;  
last=newnode;  
}  
}
```

```
void traverse(){  
if(last==NULL) return;
```

```
struct node *temp=last->next;  
do{  
printf("%d ",temp->data);  
temp=temp->next;  
}while(temp!=last->next);  
}
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
int main(){
int ch;
while(1){
printf("\n1.Insert 2.Traverse 3.Exit\n");
scanf("%d",&ch);
switch(ch){
case 1: insert(); break;
case 2: traverse(); break;
case 3: exit(0);
}
}
}
```

Output/Results:



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

4. C Program to perform Stack implementation using Arrays and ADT.

i) Using Array

```
#include<stdlib.h>
```

```
#define MAX 100
```

```
int stack[MAX],top=-1;
```

```
void push(){
```

```
int x;
```

```
if(top==MAX-1){
```

```
printf("Overflow\n");
```

```
return;
```

```
}
```

```
printf("Enter value: ");
```

```
scanf("%d",&x);
```

```
stack[++top]=x;
```

```
}
```

```
void pop(){
```

```
if(top==-1){
```

```
printf("Underflow\n");
```

```
return;
```

```
}
```

```
printf("Popped: %d\n",stack[top--]);
```

```
}
```

```
void display(){
```

```
for(int i=top;i>=0;i--)
```

```
printf("%d\n",stack[i]);
```

```
}
```

```
int main(){
```

```
int ch;
```

```
while(1){
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)

NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE

Approved by AICTE, New Delhi, Affiliated to JNTUH

```
printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
scanf("%d",&ch);
switch(ch){
case 1: push(); break;
case 2: pop(); break;
case 3: display(); break;
case 4: exit(0);
}
}
}
```

Output/Results:

ii) Using Linked List

```
#include<stdio.h>
#include<stdlib.h>
```

```
struct node{
int data;
struct node *next;
};
```

```
struct node *top=NULL;
```

```
void push(){
struct node *newnode=(struct node*)malloc(sizeof(struct node));
printf("Enter value: ");
scanf("%d",&newnode->data);
newnode->next=top;
top=newnode;
}
```

```
void pop(){
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)

NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE

Approved by AICTE, New Delhi, Affiliated to JNTUH

```
if(top==NULL){
printf("Underflow\n");
return;
}
struct node *temp=top;
printf("Popped: %d\n",temp->data);
temp=temp->next;
free(temp);
}

void display(){
struct node *temp=top;
while(temp){
printf("%d\n",temp->data);
temp=temp->next;
}
}

int main(){
int ch;
while(1){
printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
scanf("%d",&ch);
switch(ch){
case 1: push(); break;
case 2: pop(); break;
case 3: display(); break;
case 4: exit(0);
}
}
}
```

Output/Results:



5. C Program to perform Queue implementation using Arrays and ADT.

i) Using Array

```
#include<stdlib.h>
```

```
#define MAX 100
```

```
int queue[MAX], front = -1, rear = -1;
```

```
void enqueue(){
```

```
int x;
```

```
if(rear == MAX-1){
```

```
printf("Overflow\n");
```

```
return;
```

```
}
```

```
printf("Enter value: ");
```

```
scanf("%d",&x);
```

```
if(front == -1) front = 0;
```

```
queue[++rear] = x;
```

```
}
```

```
void dequeue(){
```

```
if(front == -1 || front > rear){
```

```
printf("Underflow\n");
```

```
return;
```

```
}
```

```
printf("Deleted: %d\n", queue[front++]);
```

```
}
```

```
void display(){
```

```
if(front == -1){
```

```
printf("Empty Queue\n");
```

```
return;
```

```
}
```

```
for(int i=front;i<=rear;i++)
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
printf("%d ", queue[i]);  
printf("\n");  
}
```

```
int main(){  
int ch;  
while(1){  
printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");  
scanf("%d",&ch);  
switch(ch){  
case 1: enqueue(); break;  
case 2: dequeue(); break;  
case 3: display(); break;  
case 4: exit(0);  
}  
}  
}
```

Output/Results:

ii) Using Linked List

```
#include<stdlib.h>
```

```
struct node{  
int data;  
struct node *next;  
};
```

```
struct node *front=NULL, *rear=NULL;
```

```
void enqueue(){  
struct node *newnode=(struct node*)malloc(sizeof(struct node));  
printf("Enter value: ");  
scanf("%d",&newnode->data);
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)

NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE

Approved by AICTE, New Delhi, Affiliated to JNTUH

```
newnode->next=NULL;
if(rear==NULL)
front=rear=newnode;
else{
rear->next=newnode;
rear=newnode;
}
}
void dequeue(){
if(front==NULL){
printf("Underflow\n");
return;
}
struct node *temp=front;
printf("Deleted: %d\n", temp->data);
front=front->next;
free(temp);
}
void display(){
struct node *temp=front;
while(temp){
printf("%d ", temp->data);
temp=temp->next;
}
printf("\n");
}
int main(){
int ch;
while(1){
printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
scanf("%d",&ch);
switch(ch){
case 1: enqueue(); break;
case 2: dequeue(); break;
case 3: display(); break;
case 4: exit(0); }}}}
```



Output/Results:

6. C Program to perform Sorting Techniques: Radix Sort, Heap Sort, Shell Sort, Tree Sort.

(i) Radix Sort

```
void radixSort(int arr[], int n){
int bucket[10][10], count[10];
int i,j,k,rem,div=1;

for(int pass=0; pass<3; pass++){
for(i=0;i<10;i++) count[i]=0;

for(i=0;i<n;i++){
rem=(arr[i]/div)%10;
bucket[rem][count[rem]++]=arr[i];
}

i=0;
for(k=0;k<10;k++)
for(j=0;j<count[k];j++)
arr[i++]=bucket[k][j];

div*=10;
}
}

int main(){
int arr[]={170,45,75,90,802};
int n=5;

radixSort(arr,n);
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)

NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE

Approved by AICTE, New Delhi, Affiliated to JNTUH

```
for(int i=0;i<n;i++)  
printf("%d ",arr[i]);  
}
```

Output/Results:

(ii) Heap Sort

```
void heapify(int arr[], int n, int i){  
int largest=i;  
int l=2*i+1;  
int r=2*i+2;
```

```
if(l<n && arr[l]>arr[largest])  
largest=l;
```

```
if(r<n && arr[r]>arr[largest])  
largest=r;
```

```
if(largest!=i){  
int temp=arr[i];  
arr[i]=arr[largest];  
arr[largest]=temp;
```

```
heapify(arr,n,largest);  
}  
}
```

```
void heapSort(int arr[], int n){  
for(int i=n/2-1;i>=0;i--)  
heapify(arr,n,i);
```

```
for(int i=n-1;i>0;i--){  
int temp=arr[0];  
arr[0]=arr[i];
```




```
arr[i]=temp;
```

```
heapify(arr,i,0);  
}  
}
```

```
int main(){  
int arr[]={4,10,3,5,1};  
int n=5;
```

```
heapSort(arr,n);
```

```
for(int i=0;i<n;i++)  
printf("%d ",arr[i]);  
}
```

Output/Results:

(iii) Shell Sort

```
#include<stdio.h>
```

```
void shellSort(int arr[], int n){  
    for(int gap=n/2;gap>0;gap/=2){  
        for(int i=gap;i<n;i++){  
            int temp=arr[i];  
            int j;  
            for(j=i;j>=gap && arr[j-gap]>temp;j-=gap)  
                arr[j]=arr[j-gap];  
  
            arr[j]=temp;  
        }  
    }  
}
```

```
int main(){  
    int arr[]={12,34,54,2,3};
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
int n=5;

shellSort(arr,n);

for(int i=0;i<n;i++)
    printf("%d ",arr[i]);
}
```

Output/Results:

(iv) Tree Sort

```
#include<stdio.h>
#include<stdlib.h>
struct node{
    int data;
    struct node *left,*right;
};
struct node* insert(struct node* root, int x){
    if(root==NULL){
        root=(struct node*)malloc(sizeof(struct node));
        root->data=x;
        root->left=root->right=NULL;
        return root;
    }
    if(x<root->data)
        root->left=insert(root->left,x);
    else
        root->right=insert(root->right,x);

    return root;
}

void inorder(struct node* root){
    if(root){
        inorder(root->left);
        printf("%d ",root->data);
        inorder(root->right);
    }
}
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)

NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE

Approved by AICTE, New Delhi, Affiliated to JNTUH

```
}  
}
```

```
int main(){  
    int arr[]={5,3,7,2,4};  
    struct node* root=NULL;  
  
    for(int i=0;i<5;i++)  
        root=insert(root,arr[i]);  
  
    inorder(root);  
}
```

Output/Results:



7. C Program to perform Tree Traversal Methods (Recursive and Non-Recursive).

```
#include<stdlib.h>
```

```
struct node{  
int data;  
struct node *left,*right;  
};
```

```
struct node* createNode(int x){  
struct node* newnode=(struct node*)malloc(sizeof(struct node));  
newnode->data=x;  
newnode->left=newnode->right=NULL;  
return newnode;  
}
```

```
/* Recursive Traversals */
```

```
void inorder(struct node* root){  
if(root){  
inorder(root->left);  
printf("%d ",root->data);  
inorder(root->right);  
}  
}
```

```
void preorder(struct node* root){  
if(root){  
printf("%d ",root->data);  
preorder(root->left);  
preorder(root->right);  
}  
}
```

```
void postorder(struct node* root){
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
if(root){
postorder(root->left);
postorder(root->right);
printf("%d ",root->data);
}
}

/* Non-Recursive Inorder using Stack */

void inorderNR(struct node* root){
struct node* stack[100];
int top=-1;

while(root || top!=-1){
while(root){
stack[++top]=root;
root=root->left;
}
root=stack[top--];
printf("%d ",root->data);
root=root->right;
}
}

int main(){
struct node* root=createNode(1);
root->left=createNode(2);
root->right=createNode(3);

printf("Recursive Inorder: ");
inorder(root);

printf("\nNon-Recursive Inorder: ");
inorderNR(root);
}
```

Output/Results:



8. C Program to perform Binary Search Tree, B-Trees, B+ Trees, AVL Trees, Red-Black Trees.

i) Binary Search Tree

```
#include<stdlib.h>

struct node{
int data;
struct node *left,*right;
};

struct node* insert(struct node* root,int x){
if(root==NULL){
root=(struct node*)malloc(sizeof(struct node));
root->data=x;
root->left=root->right=NULL;
return root;
}
if(x<root->data)
root->left=insert(root->left,x);
else
root->right=insert(root->right,x);

return root;
}

void inorder(struct node* root){
if(root){
inorder(root->left);
printf("%d ",root->data);
inorder(root->right);
}
}
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
int main(){
struct node* root=NULL;

root=insert(root,50);
insert(root,30);
insert(root,70);
insert(root,20);
insert(root,40);

printf("BST Inorder Traversal: ");
inorder(root);
}
```

Output/Results:

ii) AVL Trees,

```
#include<stdlib.h>

struct Node {
int data;
struct Node *left, *right;
int height;
};

int height(struct Node *n){
if(n==NULL) return 0;
return n->height;
}

int max(int a,int b){
return (a>b)?a:b;
}

struct Node* newNode(int data){
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)

NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE

Approved by AICTE, New Delhi, Affiliated to JNTUH

```
struct Node* node=(struct Node*)malloc(sizeof(struct Node));
node->data=data;
node->left=node->right=NULL;
node->height=1;
return node;
}
```

```
struct Node* rightRotate(struct Node *y){
struct Node *x=y->left;
struct Node *T2=x->right;

x->right=y;
y->left=T2;

y->height=max(height(y->left),height(y->right))+1;
x->height=max(height(x->left),height(x->right))+1;

return x;
}
```

```
struct Node* leftRotate(struct Node *x){
struct Node *y=x->right;
struct Node *T2=y->left;

y->left=x;
x->right=T2;

x->height=max(height(x->left),height(x->right))+1;
y->height=max(height(y->left),height(y->right))+1;

return y;
}
```

```
int getBalance(struct Node *n){
if(n==NULL) return 0;
return height(n->left)-height(n->right);
}
```




BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

}

```
struct Node* insert(struct Node* node,int data){
```

```
if(node==NULL)
return newNode(data);
```

```
if(data<node->data)
node->left=insert(node->left,data);
else if(data>node->data)
node->right=insert(node->right,data);
else
return node;
```

```
node->height=1+max(height(node->left),height(node->right));
```

```
int balance=getBalance(node);
```

```
// LL Case
if(balance>1 && data<node->left->data)
return rightRotate(node);
```

```
// RR Case
if(balance<-1 && data>node->right->data)
return leftRotate(node);
```

```
// LR Case
if(balance>1 && data>node->left->data){
node->left=leftRotate(node->left);
return rightRotate(node);
}
```

```
// RL Case
if(balance<-1 && data<node->right->data){
node->right=rightRotate(node->right);
return leftRotate(node);
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
}
```

```
return node;
```

```
}
```

```
void inorder(struct Node *root){
```

```
if(root){
```

```
inorder(root->left);
```

```
printf("%d ",root->data);
```

```
inorder(root->right);
```

```
}
```

```
}
```

```
int main(){
```

```
struct Node *root=NULL;
```

```
root=insert(root,10);
```

```
root=insert(root,20);
```

```
root=insert(root,30);
```

```
root=insert(root,40);
```

```
root=insert(root,50);
```

```
root=insert(root,25);
```

```
printf("Inorder Traversal of AVL Tree:\n");
```

```
inorder(root);
```

```
return 0;
```

```
}
```

Output/Results:

iii) Red-Black Trees

```
#include<stdlib.h>
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
#define RED 1
```

```
#define BLACK 0
```

```
struct Node {  
    int data;  
    int color;  
    struct Node *left,*right,*parent;  
};
```

```
struct Node *root=NULL;
```

```
struct Node* newNode(int data){  
    struct Node* node=(struct Node*)malloc(sizeof(struct Node));  
    node->data=data;  
    node->color=RED;  
    node->left=node->right=node->parent=NULL;  
    return node;  
}
```

```
void leftRotate(struct Node *x){  
    struct Node *y=x->right;  
    x->right=y->left;
```

```
    if(y->left!=NULL)  
        y->left->parent=x;
```

```
    y->parent=x->parent;
```

```
    if(x->parent==NULL)  
        root=y;  
    else if(x==x->parent->left)  
        x->parent->left=y;  
    else  
        x->parent->right=y;
```

```
    y->left=x;
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
x->parent=y;  
}
```

```
void rightRotate(struct Node *y){  
    struct Node *x=y->left;  
    y->left=x->right;
```

```
    if(x->right!=NULL)  
        x->right->parent=y;
```

```
    x->parent=y->parent;
```

```
    if(y->parent==NULL)  
        root=x;  
    else if(y==y->parent->left)  
        y->parent->left=x;  
    else  
        y->parent->right=x;
```

```
    x->right=y;  
    y->parent=x;  
}
```

```
void fixInsert(struct Node *z){  
    while(z->parent && z->parent->color==RED){
```

```
        if(z->parent==z->parent->parent->left){  
            struct Node *y=z->parent->parent->right;
```

```
            if(y && y->color==RED){  
                z->parent->color=BLACK;  
                y->color=BLACK;  
                z->parent->parent->color=RED;  
                z=z->parent->parent;  
            }else{  
                if(z==z->parent->right){
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
z=z->parent;
leftRotate(z);
}
z->parent->color=BLACK;
z->parent->parent->color=RED;
rightRotate(z->parent->parent);
}
}else{
struct Node *y=z->parent->parent->left;
```

```
if(y && y->color==RED){
z->parent->color=BLACK;
y->color=BLACK;
z->parent->parent->color=RED;
z=z->parent->parent;
}else{
if(z==z->parent->left){
z=z->parent;
rightRotate(z);
}
z->parent->color=BLACK;
z->parent->parent->color=RED;
leftRotate(z->parent->parent);
}
}
}
root->color=BLACK;
}
```

```
void insert(int data){
struct Node *z=newNode(data);
struct Node *y=NULL;
struct Node *x=root;
```

```
while(x!=NULL){
y=x;
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
if(z->data<x->data)
x=x->left;
else
x=x->right;
}
```

```
z->parent=y;
```

```
if(y==NULL)
root=z;
else if(z->data<y->data)
y->left=z;
else
y->right=z;
```

```
fixInsert(z);
}
```

```
void inorder(struct Node *root){
if(root){
inorder(root->left);
printf("%d ",root->data);
inorder(root->right);
}
}

int main(){
insert(10);
insert(20);
insert(30);
insert(15);
insert(25);
printf("Inorder Traversal of Red-Black Tree:\n");
inorder(root);

return 0;
```



}

Output/Results:

9. C Program to perform Graph Traversal Methods.(DFS and BFS)

```
int adj[10][10], visited[10], n;
```

```
void dfs(int v){  
    visited[v]=1;  
    printf("%d ",v);
```

```
    for(int i=0;i<n;i++){  
        if(adj[v][i]==1 && !visited[i])  
            dfs(i);  
    }  
}
```

```
void bfs(int v){  
    int queue[10], front=0, rear=0;
```

```
    visited[v]=1;  
    queue[rear++]=v;
```

```
    while(front<rear){  
        v=queue[front++];  
        printf("%d ",v);
```

```
        for(int i=0;i<n;i++){  
            if(adj[v][i]==1 && !visited[i]){  
                visited[i]=1;  
                queue[rear++]=i;  
            }  
        }  
    }  
}
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
int main(){
int start;

printf("Enter number of vertices: ");
scanf("%d",&n);

printf("Enter adjacency matrix:\n");
for(int i=0;i<n;i++)
for(int j=0;j<n;j++)
scanf("%d",&adj[i][j]);

printf("Enter starting vertex: ");
scanf("%d",&start);

printf("DFS: ");
dfs(start);

for(int i=0;i<n;i++)
visited[i]=0;

printf("\nBFS: ");
bfs(start);
}
```

Output/Results:



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

10. C Program to perform Hashing Techniques: Division, Multiplication, Mid-Square, Folding.

```
#include<math.h>
```

```
/* Division Method */  
int divisionHash(int key, int size){  
    return key % size;  
}
```

```
/* Multiplication Method */  
int multiplicationHash(int key, int size){  
    float A = 0.618;  
    float temp = key * A;  
    return (int)(size * (temp - (int)temp));  
}
```

```
/* Mid-Square Method */  
int midSquareHash(int key, int size){  
    int square = key * key;  
    return (square / 10) % size;  
}
```

```
/* Folding Method */  
int foldingHash(int key, int size){  
    int sum = 0;  
    while(key > 0){  
        sum += key % 100;  
        key /= 100;  
    }  
    return sum % size;  
}
```

```
int main(){
```



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

```
int key = 1234;
```

```
int size = 10;
```

```
printf("Division: %d\n", divisionHash(key,size));  
printf("Multiplication: %d\n", multiplicationHash(key,size));  
printf("Mid-Square: %d\n", midSquareHash(key,size));  
printf("Folding: %d\n", foldingHash(key,size));  
}
```

Output/Results:



BHARAT

INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution - UGC, GOVT. of INDIA)
NAAC Accredited with 'A' Grade, NBA Accredited for UG Programmes: CSE, ECE
Approved by AICTE, New Delhi, Affiliated to JNTUH

TEXT BOOKS:

1. Fundamentals of Data Structures in C, 2nd Edition, E. Horowitz, S. Sahni and Susan Anderson Freed, Universities Press.
2. Data Structures using C – A. S. Tanenbaum, Y. Langsam, and M. J. Augenstein, PHI/Pearson Education.

REFERENCE BOOK:

1. Data Structures: A Pseudocode Approach with C, 2nd Edition, R. F. Gilberg and B. A. Forouzan, Cengage Learning.